

Neural Networks as Continuous Dynamical Systems

A Study Using Differential Equations

Somesh Biswas
Roll: BSD-CC-2420

Indian Statistical Institute, Kolkata
Mathematics IV Project

May 7, 2026

Introduction: The Core Observation

- A standard Deep Residual Network (ResNet) update:

$$h_{t+1} = h_t + f(h_t, \theta_t)$$

- A forward Euler step for numerical integration:

$$h_{n+1} = h_n + \Delta t \cdot f(h_n, t_n)$$

- **Key Insight:** Setting $\Delta t = 1$ makes these equations identical. Stacking residual blocks is implicitly running a coarse numerical integrator on a continuous system.

Initial Value Problem (IVP)

- An ODE describes the evolution of a variable over continuous time.
- Formulated as:

$$\frac{dh(t)}{dt} = f(h(t), t), \quad h(t_0) = h_0$$

- Forward Euler Method approximates this via finite differences:

$$h_{n+1} = h_n + \Delta t \cdot f(h_n, t_n)$$

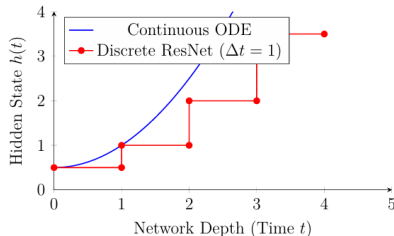
The Continuous Limit: Neural ODEs

- Instead of treating layer depth as a discrete index, we treat it as a continuous variable $t \in [0, T]$.
- As $\Delta t \rightarrow 0$, the discrete update converges to a first-order ODE:

$$\frac{dh(t)}{dt} = f(h(t), \theta(t), t)$$

- **Forward Pass:** Identical to solving an IVP from $t = 0$ to $t = T$.

Figure 1 (Discrete vs Continuous Trajectory)



Picard-Lindelöf Theorem

If f is uniformly Lipschitz continuous in h , there exists a unique solution $h(t)$ to the IVP.

Why does this matter for Deep Learning?

- Guarantees the network's output is deterministic.
- Depends on the activation function:
 - **Tanh**: Derivative bounded by 1. Lipschitz constant $L = 1$.
 - **ReLU**: Non-differentiable at 0, but uniformly Lipschitz continuous ($L = 1$).
- Continuous networks can safely use modern, non-smooth activations.

Training: Adjoint Sensitivity Method

- Standard backpropagation fails for continuous ODEs. We reformulate training as a Boundary Value Problem (BVP).
- We define the adjoint state $a(t)$ as the loss gradient:

$$a(t) = \frac{\partial L}{\partial h(t)}$$

- **Adjoint ODE (Backward Pass):**

$$\frac{da(t)}{dt} = -a(t)^T \frac{\partial f(h(t), \theta, t)}{\partial h}$$

- Gradients are computed by integrating backward from $t = T$ to $t = 0$.

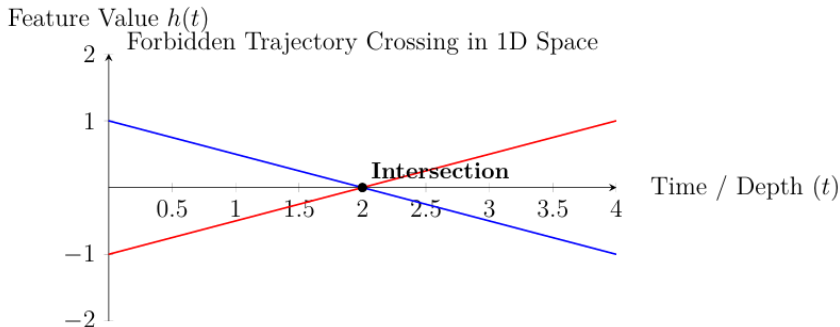
Open Problem: Topological Constraints

- Continuous networks face strict limits on expressivity.
- Because solutions to Lipschitz-continuous ODEs are unique, **trajectories in phase space cannot cross**.
- The network acts as a homeomorphism (a smooth deformation of input space).
- **The Limitation:** It cannot swap two points, break a loop, or disconnect regions.

Visualizing the Topological Constraint

- Consider a 1D mapping: $-1 \rightarrow 1$ and $1 \rightarrow -1$.
- The trajectory from -1 must cross the trajectory from 1 . Picard's Theorem forbids this intersection.

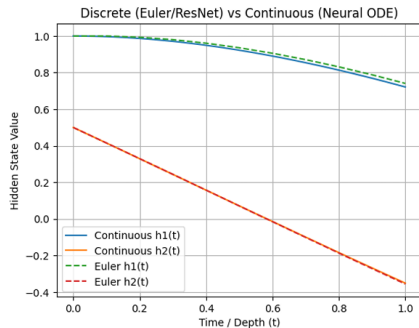
Figure 2 (Forbidden Trajectory Crossing)



Computational Demonstration

- Comparing a discrete Euler step ($\Delta t = 1$) vs. 'solve;vp'(RK45).
- Euler approximations follow a stepwise path and deviate depending on underlying system dynamics.

Figure 3 (Discrete vs Continuous Graph)



- The ResNet-to-ODE connection is foundational, not just a metaphor.
- **Theoretical Rigor:** Picard-Lindelöf guarantees well-behaved deterministic forward passes.
- **Practical Optimization:** The adjoint method allows memory-efficient, continuous gradient computation.
- **Hard Limits:** Geometric constraints of ODEs dictate what models can learn, requiring dimensional lifting (Augmented Neural ODEs) to solve crossing issues.
- Differential equations provide the rigorous language to define the exact capabilities of these deep learning models.